

ShopNest Frontend Learning & Architecture Guide

Welcome! Building your first full-stack React frontend connected to an Express backend is a major milestone. This guide provides a conceptual overview, architecture blueprints, custom hooks, and a list of backend items you'll need to configure or fix to successfully connect your frontend.

🔧 Step 1: Frontend Setup & Dependencies

Your Vite project (`Shopnest_Frontend`) currently only has `react` and `react-dom` installed. To build a fully-functional eCommerce app, you should install several essential packages.

Run the following command in your `Shopnest_Frontend` directory:

```
npm install react-router-dom @reduxjs/toolkit react-redux lucide-react
```

Purpose of these libraries:

- `react-router-dom` : Handles page navigation without refreshing the browser (Single Page Application routing).
- `@reduxjs/toolkit` & `react-redux` : Ideal for global state management where complex logic is involved, such as the shopping cart (adding, removing, persistent storage, quantity updating).
- `lucide-react` : A library of clean, modern icons (shopping carts, user profiles, search bar, etc.).

Suggested Directory Structure

Organize your `src` directory like this to keep your code clean and modular:

```
Shopnest_Frontend/src/
├── assets/           # Images, static logos, illustrations
├── components/       # Reusable presentation components
│   ├── Navbar.jsx   # Header with logo, navigation links, and cart icon
│   ├── Footer.jsx   # Footer with links and copyright
│   ├── ProductCard.jsx # Card showcasing image, price, title, "Add to Cart" button
│   └── ProtectedRoute.jsx # Wrapper component to secure authenticated routes
├── context/          # Global React Contexts
│   └── AuthContext.jsx # Authentication state and logic (Login/Logout/2FA)
├── hooks/            # Reusable custom React hooks
│   └── useFetch.js   # Custom fetch hook for GET requests
├── pages/            # Page-level components (mapped to routes)
│   ├── Home.jsx     # Landing page with product lists
│   ├── ProductDetail.jsx # Individual product page showing details
│   ├── Login.jsx     # Login form with email/password and OTP/2FA support
│   ├── Register.jsx  # Signup form that handles OTP verification
│   └── Cart.jsx      # Shopping cart view with item list and checkout button
```

🔧 Step 2: API Integration & Vite Proxy (CORS Setup)

```
fetch('/api/product')
```

Vite will proxy this request to `http://localhost:5000/api/product`, completely avoiding CORS issues in development!

Step 3: Creating your Custom Hook: `useFetch`

A custom hook is a Javascript function whose name starts with `use` and can call other hooks. Creating a `useFetch` hook isolates your data-fetching logic so you don't repeat boilerplate code (`useState`, `useEffect`) on every page.

Here is the design for `useFetch` with explanations of why each part is required:

```
import { useState, useEffect } from 'react';

/**
 * Custom hook for GET requests
 * @param {string} url - API endpoint to fetch
 * @param {object} options - Fetch options (headers, etc.)
 */
export const useFetch = (url, options = {}) => {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // 1. Create flags to prevent race conditions and memory leaks
    let isMounted = true;

    // 2. AbortController allows us to cancel the request if the component unmounts
    // or if the URL changes before the previous response has finished loading.
    const controller = new AbortController();

    const fetchData = async () => {
      setIsLoading(true);
      try {
        const response = await fetch(url, {
          ...options,
          signal: controller.signal,
        });

        if (!response.ok) {
          throw new Error(`HTTP Error! Status: ${response.status}`);
        }

        const json = await response.json();

        // Only update state if the component is still mounted
        if (isMounted) {
          setData(json);
          setError(null);
        }
      } catch (err) {
        if (isMounted && err.name !== 'AbortError') {
          setError(err.message || 'Something went wrong');
        }
      } finally {

```

```

        if (isMounted) {
            setIsLoading(false);
        }
    }
};

fetchData();

// 3. Cleanup function runs when URL changes or component unmounts
return () => {
    isMounted = false;
    controller.abort();
};
}, [url]); // Dependency array: re-run fetch whenever the URL changes

return { data, error, isLoading };
};

```

Why do we need the cleanup functions?

1. **isMounted = false** : If a user clicks away from a page while it's fetching, the component unmounts. If you try to update the state (**setData** , **setIsLoading**) on an unmounted component, React will complain, and it causes memory leaks.
2. **AbortController** : If a search query updates rapidly, multiple fetches trigger. Aborting previous fetches ensures only the latest request's response is applied, resolving race conditions.

Step 4: Authentication & The Auth Provider Pattern

Why is an Auth Provider needed?

React has a unidirectional data flow. If login status is kept inside the **Login.jsx** component, the **Navbar.jsx** component won't know if the user is logged in, and the **Cart.jsx** component won't know where to submit the order.

The **Auth Provider** uses the **React Context API** to expose authentication states (**user** , **token** , **isAuthenticated** , **login** , **logout**) globally. Any component in the app can subscribe to this context to retrieve user status or perform actions.

Implementation Concept: **AuthContext.jsx**

```

import React, { createContext, useState, useEffect, useContext } from 'react';

// Create the Context object
const AuthContext = createContext(null);

// Provide Context to the App
export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);

```

```

const [token, setToken] = useState(null);
const [loading, setLoading] = useState(true);

// Read saved session on app load (avoids logouts on refresh)
useEffect(() => {
  const storedUser = localStorage.getItem('user');
  const storedToken = localStorage.getItem('token');

  if (storedUser && storedToken) {
    setUser(JSON.parse(storedUser));
    setToken(storedToken);
  }
  setLoading(false);
}, []);

// Login handler
const loginUser = (userData, userToken) => {
  setUser(userData);
  setToken(userToken);
  localStorage.setItem('user', JSON.stringify(userData));
  localStorage.setItem('token', userToken);
};

// Logout handler
const logoutUser = () => {
  setUser(null);
  setToken(null);
  localStorage.removeItem('user');
  localStorage.removeItem('token');
};

const value = {
  user,
  token,
  loading,
  isAuthenticated: !!user,
  isAdmin: user?.role === 'admin',
  login: loginUser,
  logout: logoutUser
};

return (
  <AuthContext.Provider value={value}>
    {!loading && children}
  </AuthContext.Provider>
);
};

// Custom Hook to consume Auth Context easily
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};

```

Wrapping the Application

Wrap your router or App layout inside `main.jsx` with the `AuthProvider` so everyone has access to it:

```
import { AuthProvider } from '../context/AuthContext';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </StrictMode>
);
```

Consuming Auth in a Component (e.g., `Navbar.jsx`)

```
import { useAuth } from '../context/AuthContext';

export default function Navbar() {
  const { isAuthenticated, user, logout } = useAuth();

  return (
    <nav>
      <span>ShopNest</span>
      {isAuthenticated ? (
        <div>
          <span>Welcome, {user.name}</span>
          <button onClick={logout}>Logout</button>
        </div>
      ) : (
        <a href="/login">Login</a>
      )}
    </nav>
  );
}
```

🔒 Step 5: Route Protection (`ProtectedRoute.jsx`)

You must protect checkout and admin pages from unauthorized users. If a guest tries to access `/checkout` or `/admin`, redirect them to `/login`.

Create a wrapper component `ProtectedRoute.jsx`:

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
```

```

export const ProtectedRoute = ({ adminOnly = false }) => {
  const { isAuthenticated, isAdmin, loading } = useAuth();

  if (loading) return <div>Loading authentication...</div>;

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (adminOnly && !isAdmin) {
    return <Navigate to="/" replace />; // Non-admins kicked to homepage
  }

  // Outlet renders child routes defined in App.jsx
  return <Outlet />;
};

```

In your `App.jsx`, configure your routing tree:

```

import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import { ProtectedRoute } from './components/ProtectedRoute';
// Page imports... (Home, Login, Cart, AdminDashboard, etc.)

const router = createBrowserRouter([
  // --- Public Routes ---
  { path: "/", element: <Home /> },
  { path: "/product/:id", element: <ProductDetail /> },
  { path: "/login", element: <Login /> },
  { path: "/signup", element: <Register /> },

  // --- User Protected Routes ---
  {
    element: <ProtectedRoute />, // This acts as the wrapper layout
    children: [
      { path: "/cart", element: <Cart /> },
      { path: "/orders", element: <Orders /> },
    ],
  },

  // --- Admin Only Routes ---
  {
    element: <ProtectedRoute adminOnly={true} />, // Wrapper layout with props
    children: [
      { path: "/admin", element: <AdminDashboard /> },
    ],
  },
]);

function App() {
  // Simply return the provider with our router configuration
  return <RouterProvider router={router} />;
}

export default App;

```

Step 6: Good Practices vs. Bad Practices

Good Practices

- **Handle authorization headers uniformly:** In custom requests (like POST or PUT orders/products), retrieve the token from the Auth Context and put it in the `Authorization` header: `Authorization: Bearer <token>`.
- **Form Validation:** Always validate user input on the frontend before submitting to endpoints. Check for password length, matching confirmation fields, and correct email formats to reduce API load and improve UI latency.
- **Handling File Uploads (Multer):** The backend product creation utilizes `multer` for images. On the frontend, you **cannot** send standard JSON for `POST /api/product`. You must use `FormData` object:

```
const formData = new FormData();
formData.append('name', name);
formData.append('price', price);
formData.append('image_url', fileObject); // fileObject from <input type="file" />

// Fetch call: Do NOT specify Content-Type header when sending FormData;
// the browser will automatically compute it with boundary strings.
await fetch('/api/product', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
  body: formData
});
```

- **Redux for Cart:** Use Redux slices for the cart because the cart is modified by clicking product cards, detail views, and cart views. Sharing cart operations globally across independent views is Redux's strong suit.

Bad Practices

- **Hardcoding backend URLs:** Never hardcode `http://localhost:5000` inside your fetch targets. Use Vite's `/api` proxy. This ensures your code works immediately when deployed.
- **Prop Drilling:** Passing authentication states, user data, or tokens through multiple parent-child components is bad. Use the Auth Context hook.
- **Leaking sensitive info:** Never store sensitive passwords in states or console logs. Ensure logout clears local storage fully.
- **Ignoring Fetch States:** Don't assume calls succeed. Always display loaders (`isLoading`), empty states, and errors to prevent a blank page or freezing UI.

Step 7: Critical Backend Analysis & Checklist

While checking your backend codebase, I found a few discrepancies that will break the frontend or cause logic gaps. You should review and tweak these backend files:

1. Token Signature Discrepancy (Major Bug)

- **In [authController.js:L21](file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/controller/authController.js#L21):** The OTP verification signs the token using the *entire user object*: `const token = generateToken(user.toObject())`
- **In [authController.js:L92](file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/controller/authController.js#L92):** The normal password login signs the token using the *user's ID*: `UserObj.token = generateToken(User._id)`
- **Why it matters:** In the authorization middleware `protect.js`, the code decodes the token and calls `userModel.findById(decoded.id)`. For tokens generated during OTP verification, `decoded.id` will be the entire user object instead of a string ID, causing database lookup errors and authorization failures on subsequent calls.
- **Recommended Action:** Update `authController.js` so that `verify_otp` generates the token using `user._id` (matching `postLogin`):

```
const token = generateToken(user._id)
```

2. User Order Endpoint (GET /api/order/my-orders)

- **In [orderController.js:L15-L28](file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/controller/orderController.js#L15-L28):** The controller retrieves a specific order using `req.body._id`.
- **Why it matters:**
 - The endpoint is a `GET` route (`OrderRouter.get("/my-orders", getMyOrder)`). Standard HTTP clients (like `fetch` or `axios`) do not support or strip out Request Bodies in `GET` requests.
 - The route does **not** have the `protect` middleware attached in `orderRoute.js`. This allows unauthenticated users to fetch order documents, representing a security vulnerability.
- **Recommended Action:** Change the order routing to look up order histories using the authenticated user ID (`req.user._id`) from the `protect` middleware:

```
// Route: OrderRouter.get('/my-orders', protect, getMyOrders)
// Controller:
exports.getMyOrders = async (req, res) => {
  try {
    const orders = await orderModel.find({ user: req.user._id });
    res.json({ orders });
  } ...
}
```

3. Placing Orders requires Admin (Auth Check Bug)

- In [orderRoute.js:L5](file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/routes/orderRoute.js#L5):

```
OrderRouter.route('/').get(protect,IsAdmin ,getAllOrders).post(protect,IsAdmin ,createOrder).put(protect,IsAdmin ,updateOrderStatus)
```
- **Why it matters:** The `POST` route to submit orders requires the `IsAdmin` middleware. Normal customers (role: "user") trying to submit an order will receive a `401 Unauthorized` response.
- **Recommended Action:** Separate user routes from admin routes:

```
OrderRouter.route('/')
  .get(protect, IsAdmin, getAllOrders) // Admin gets all orders
  .post(protect, createOrder)          // Any logged-in user can place an order
  .put(protect, IsAdmin, updateOrderStatus); // Admin updates order status
```

4. Commented Payment Router

- In [app.js:L18](file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/app.js#L18): `// app.use("/api/payment" ,PaymentRouter )` is commented out.
- **Why it matters:** When you attempt to set up the checkout flow with Razorpay payments, calls to `/api/payment/...` will return 404 status codes until this line is uncommented and the route is mounted.

5. Missing CORS Middleware

- Although you have `cors` in your dependencies, it is not imported or used in [app.js] (file:///c:/Users/ak939/OneDrive%20-%20RIL%20INDIA%20ANASH/MERN/PROJECTS/ShopNest/Backend/app.js).
- If you run the frontend and backend on different ports, it's best to apply `app.use(cors())` at the top of your backend middleware list to allow headers, cookies, and resource sharing.